CSC1109 – Data (Analytics) at Speed and Scale

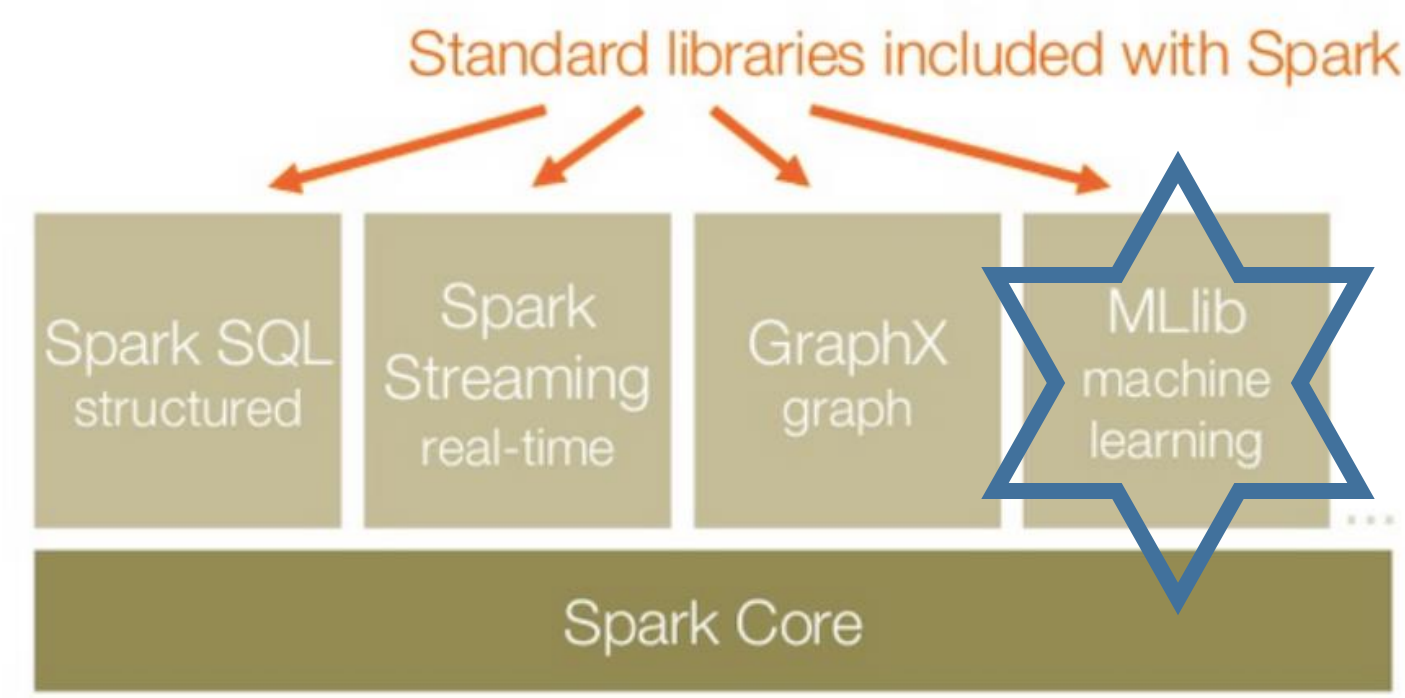
Alessandra Mileo

alessandra.mileo@dcu.ie

Our Spark journey

- Part 1: Spark Introduction
- Part 2: RDDs
- Part 3: Using Spark
- Part 4: Machine Learning with Spark
  - ML tasks in Spark: brief intro
  - DataFrames and DataSets in the context of ML

Machine Learning with Spark



MLlib features

- Multi-language support
- Scalable
- Simple to deploy
- Support for dense/regular as well as sparse data
- Supports many ML tasks
- spark.mllib and spark.ml

MLlib Machine Learning tasks

- Basic statistics
- Regression
- Classification
- Clustering
- Decomposition
- Recommendation

Data types

- Dense (regular) and sparse vectors (*):

```
val denseV: Vector = Vectors.dense(1.0, 0.0, 3.0)
val sparseV: Vector = Vectors.sparse(3, Array(0, 2), Array(1.0, 3.0))
```

- Labeled vectors (*):

```
val positiveVector = LabeledPoint(1.0, Vectors.dense(1.0, 0.0, 3.0))
```

- Non-distributed dense and sparse matrixes (*):

```
val dm: Matrix = Matrices.dense(3, 2, Array(1.0, 3.0, 5.0, 2.0, 4.0, 6.0))
```

- Various distributed (RDD-type) matrixes:

```
val rows: RDD[Vector] = ... // an RDD of vectors
val matrix: RowMatrix = new RowMatrix(rows)
```

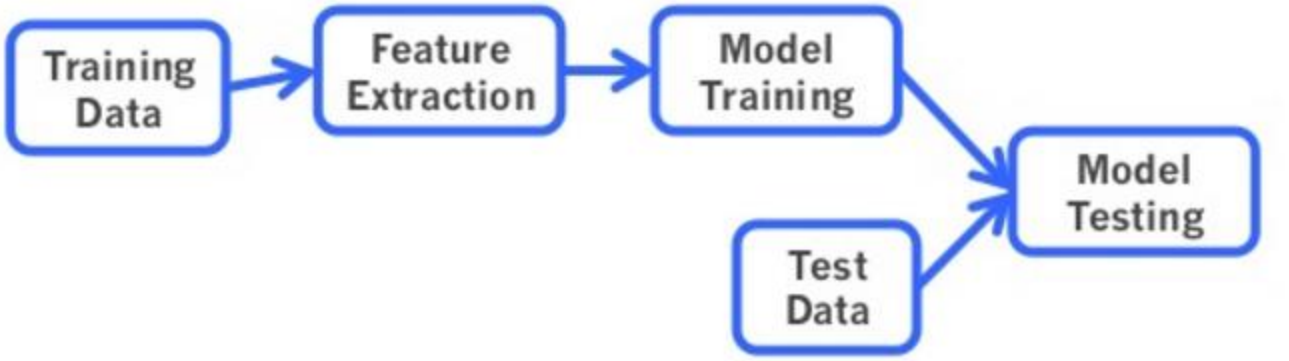
(*) wrap into RDD with, e.g. JavaRDD<Vector>

See more on Data-Types documentation: https://spark.apache.org/docs/2.3.0/mllib-data-types.html

MLlib basic statistics

- Spark allows to compute various basic statistics:
  - Summary statistics (includes total count, max, min, mean, variance)
  - Correlations
  - Stratified sampling
  - Hypothesis testing
  - Kernel density estimation
  - Random data generation

Typical ML workflow



Supervised Learning with MLlib

- Learn a function that makes prediction for us:
  - Estimate a function f(x) so that y=f(x)
- Supported approaches
  - Regression
  - Classification
- Some approaches not supported

ML with Spark

- Supported ML methods in SPARK
  - SVM
  - Logistic regression
  - Decision tree learning
- Training data = RDD<LabeledPoints>
- Labels = category identifiers

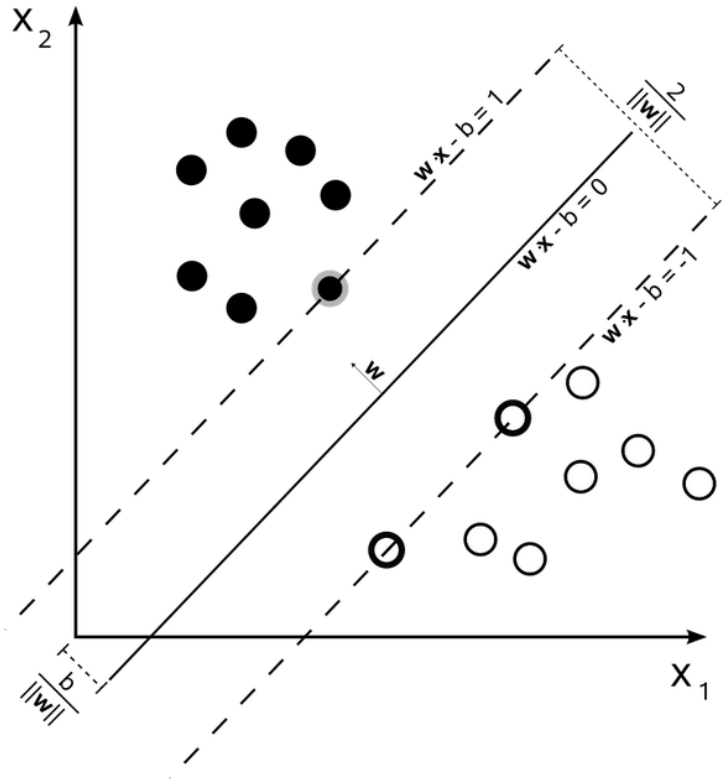
SVM in brief

- A brief look at Support Vector Machines (SVMs) first…

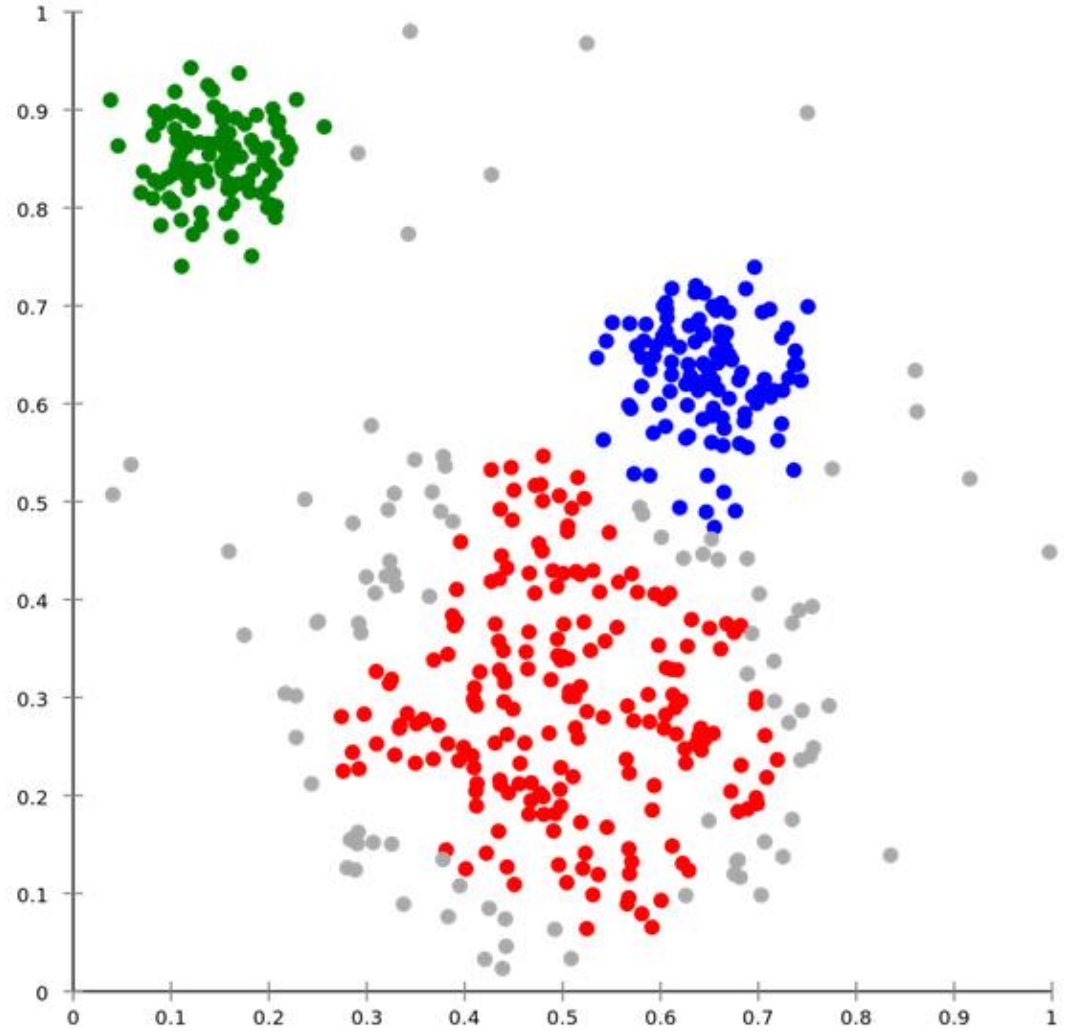
- Training dataset: $(x_1, y_1), \cdots, (x_n, y_n)$
- Each x_i is a vector of real-valued features $< x_{i1}, x_{i2}, \cdots, x_{im} >$
- Class labels $y_i \in \{1, -1\}$
- `new LabeledPoint(label`$_Y$`, featureVector`$_X$`)`

SVM in brief

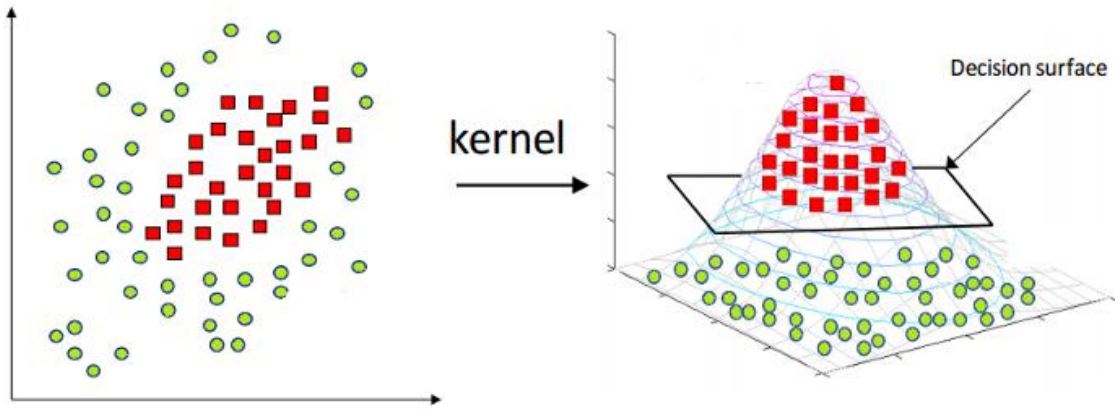
- **maximum-margin hyperplane** which separates the data items into two classes (y=-1 or y=1)
- Hyperplane: set of points x satisfying equation w • x - b = 0

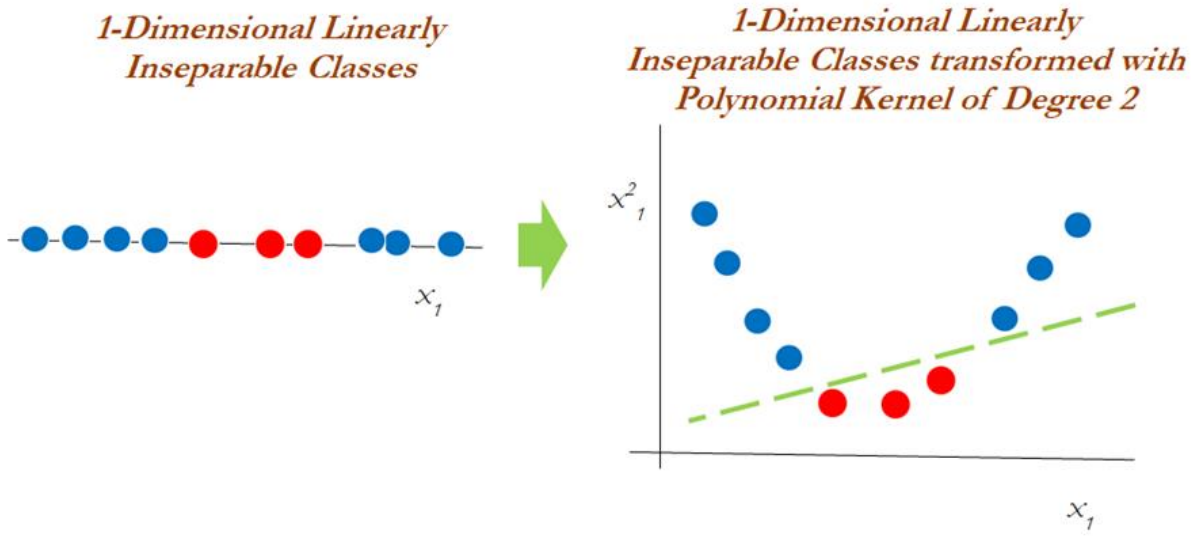


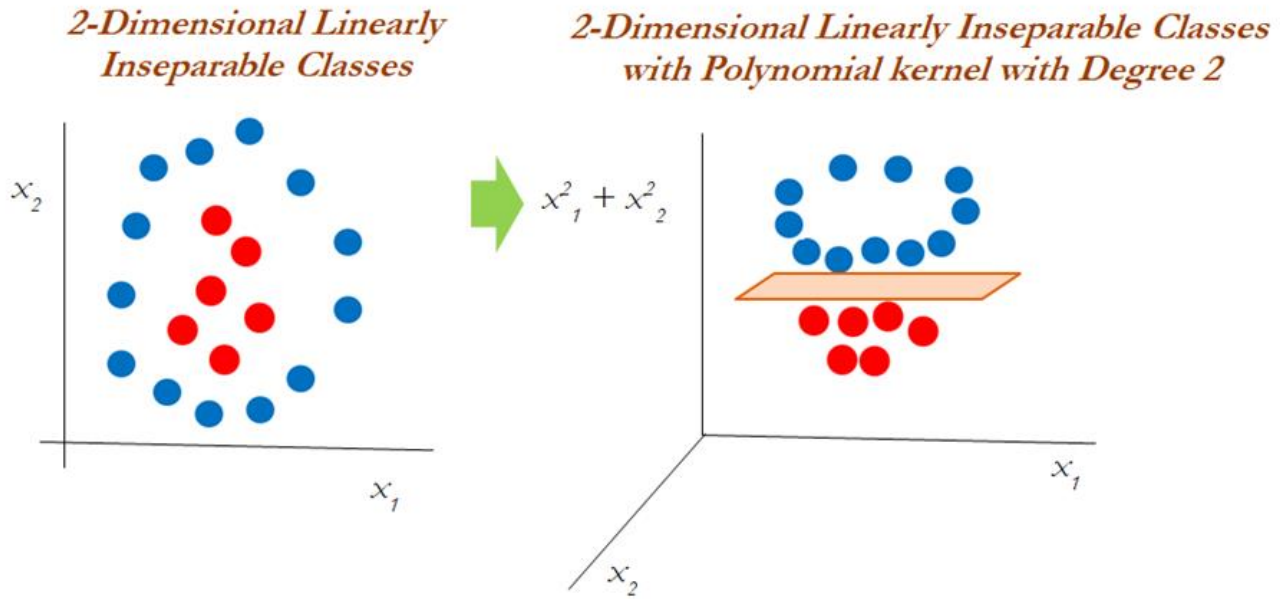
Linear SVM



SVM – Kernel trick idea







Logistic regression

- Basic idea: we learn a function of the form

$$P(y=1|x) = h_\theta(x) = \frac{1}{1+\exp(-\theta^\top x)} \equiv \sigma(\theta^\top x),$$

- Achieved by finding the "best" θ, which is done by minimizing cost function

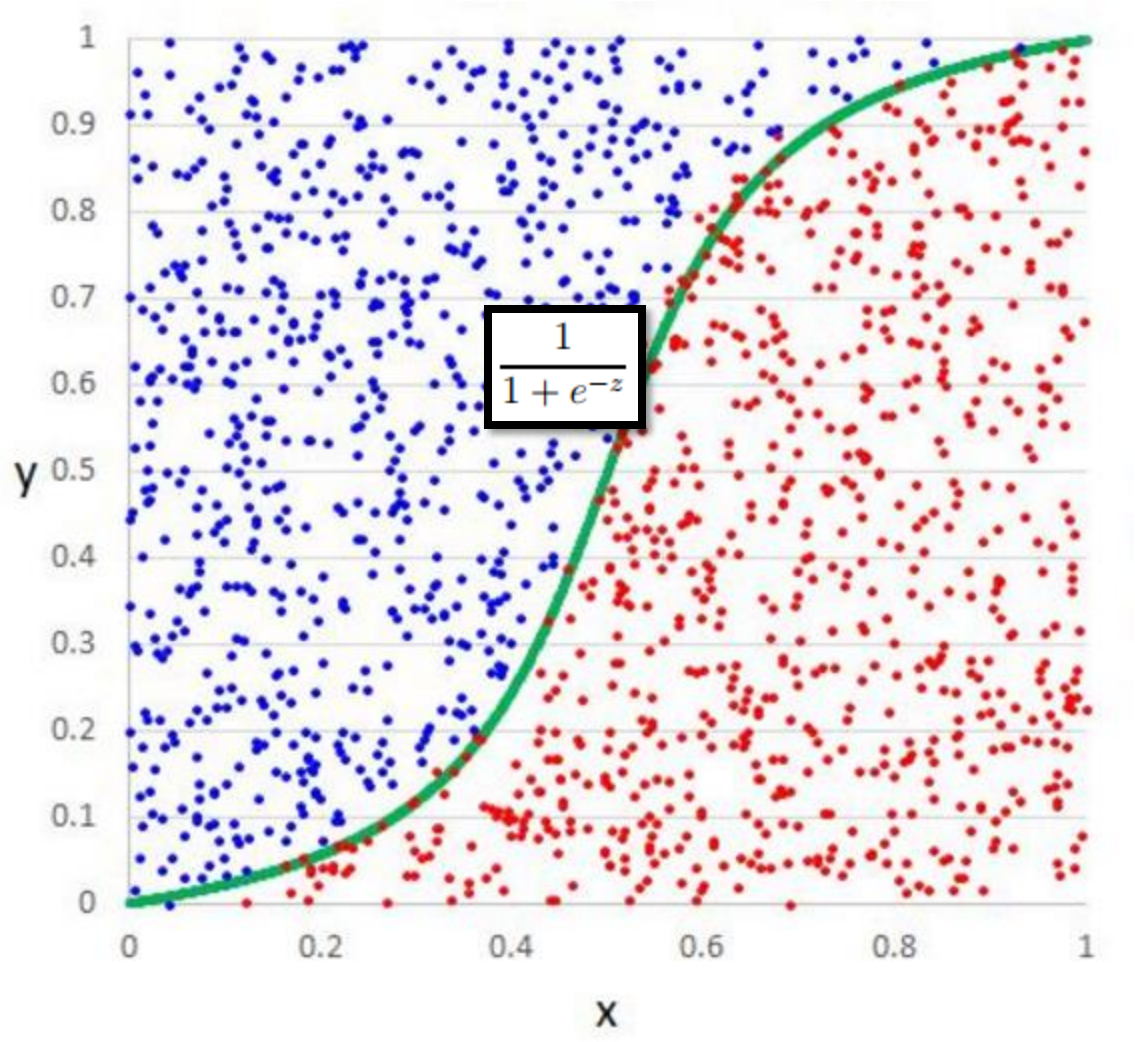
$$-\sum_i \left(y^{(i)} \log(h_\theta(x^{(i)})) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)})) \right)$$

for labelled training data (examples) $\{(x^{(i)}, y^{(i)}) : i = 1, \ldots, m\}$

- Minimisation done via mini-batch Gradient Descent or Broyden–Fletcher–Goldfarb–Shanno (BFGS)

Logistic Regression model

Logistic Regression model



Parallel ML in Spar: example

Parallel mini-batch Gradient Descent

- Gradient descent step (where J(θ) is the cost function): $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$
- Often, this takes the following form involving a sum:
- E.g.,

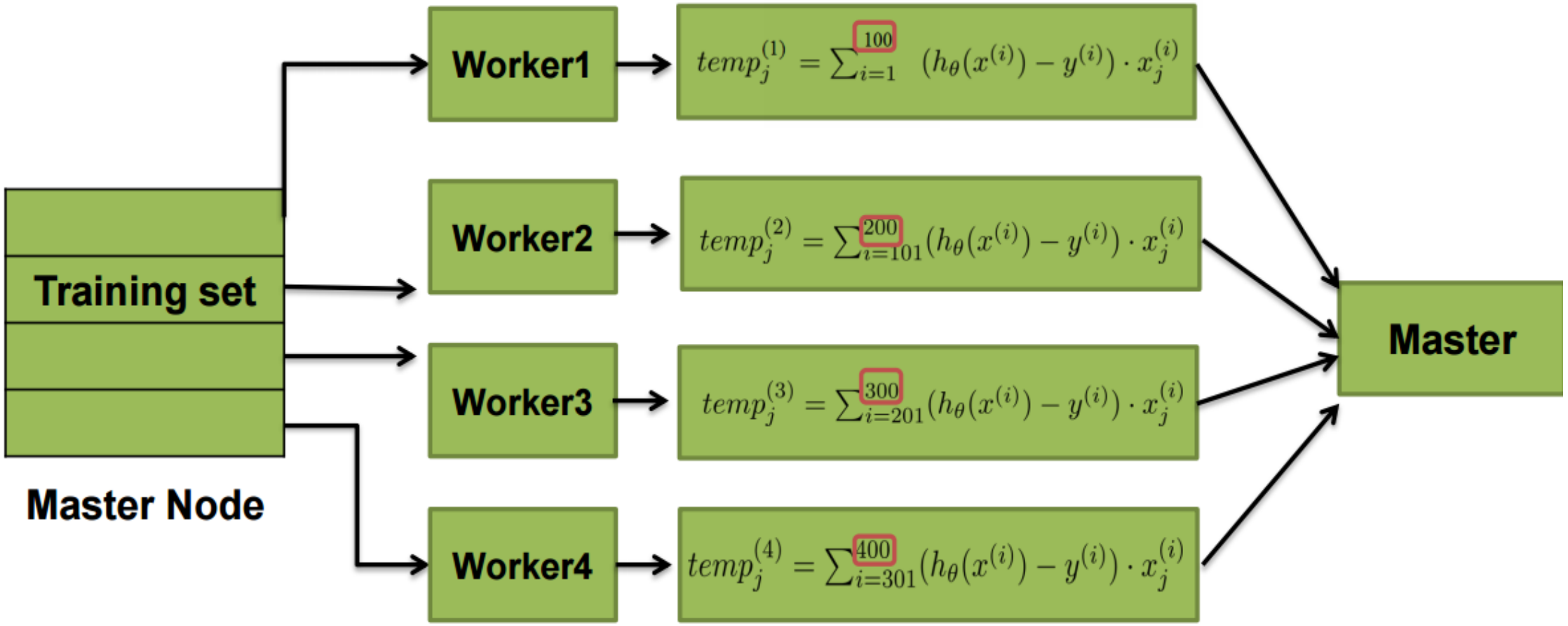
Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)} \qquad \text{(for every } j\text{)}.$$

}

Just an example - works with other gradient formulas too, if they involve a sum

Parall gradient descent



$$\theta_j := \theta_j - \alpha \frac{1}{400}(temp_j^{(1)} + temp_j^{(2)} + temp_j^{(3)} + temp_j^{(4)})$$

Hui Li: Parallel Implementation of Gradient Descent

Parallel gradient descent in Spark

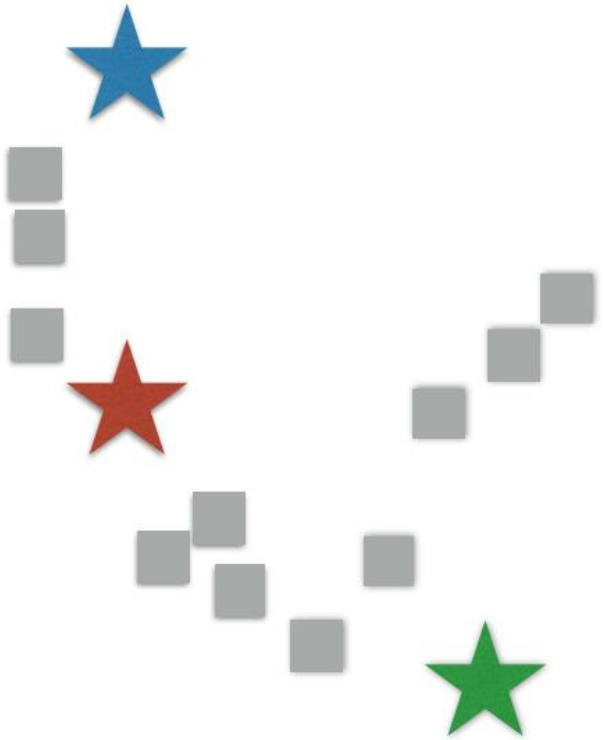
- **While not converged**
  - Broadcast model (e.g., weight vector) to the worker nodes
  - Mapper (MANY mappers)
    - Compute partial gradient for each training example
    - Combine in memory
    - Finally Yield the partial gradient
  - Reducer (single Reducer)
    - Aggregate partial gradients
    - Yield full gradient
  - Update weight vector
  - Check for convergence
- **End-While**

Unsupervised Learning with Spark

- Example: *k-means clustering*
- Easy, fast
- Basic idea:
  - Identify K (number of clusters)
  - associate observed data with nearest *means*
  - update means

K-means clustering

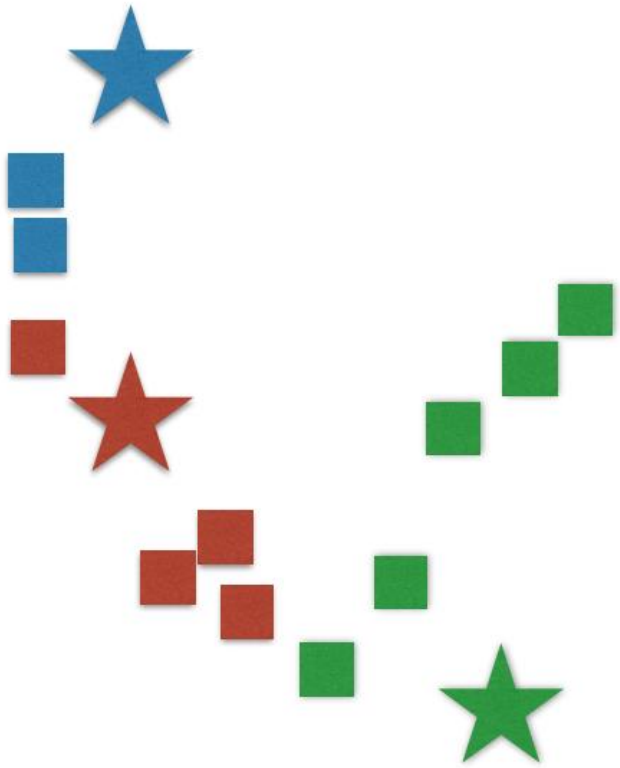
- Choose initial means (cluster centres):



Graphics by Databricks

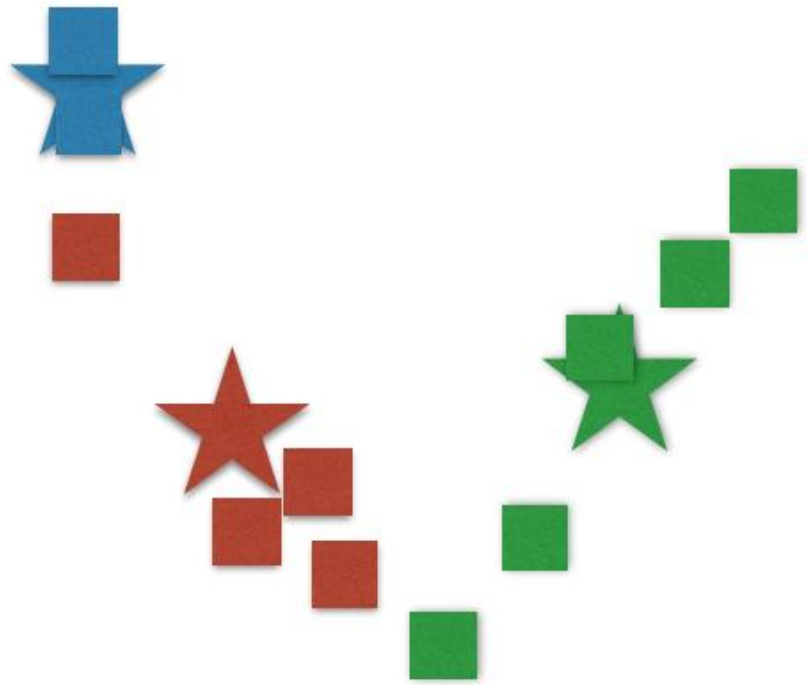
K-means clustering

- Assign observed data to nearest means:



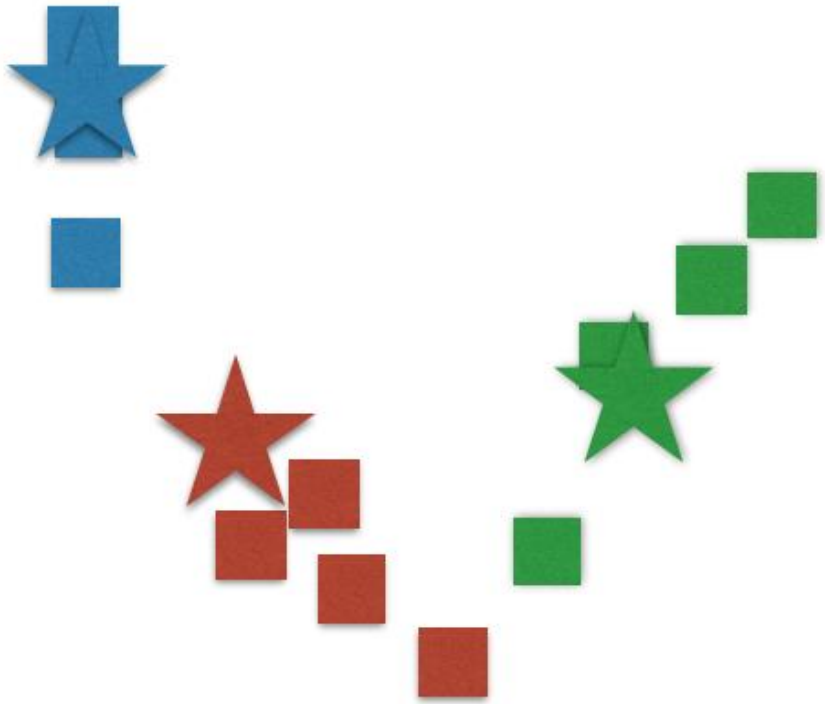
K-means clustering

- Update means



K-means clustering

- Update association of data to their nearest means (then next iteration or stop)



End: ML with Spark